

COMPUTACIÓN DE ALTAS PRESTACIONES (CAP)

Tema 2: Introducción a la Computación de Altas Prestaciones

Dr. Héctor Martínez Pérez
el2mapeh@uco.es

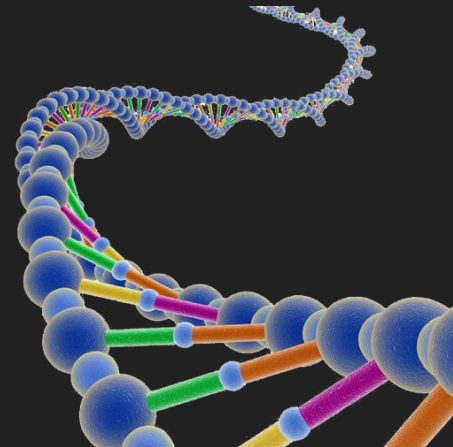
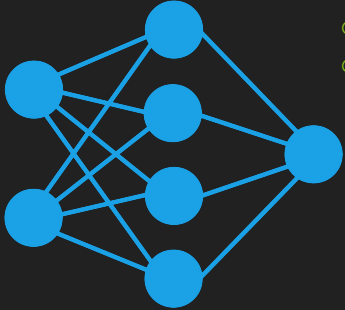
Dpto. Ingeniería Electrónica y de Computadores
Universidad de Córdoba



Necesidades

► ¿Por qué surge la necesidad de la Computación de Altas Prestaciones?

- Incremento de la demanda de prestaciones y velocidad de cómputo de los sistemas
- Necesidad de resolver problemas complejos
 - CNN (Convolutional Neural Network)
 - Alineamiento de secuencias de ADN/ARN
 - Clientes/Servidores (Google, Twitter, Google chrome, etc.)



► Ejemplo actual: chatGPT

- Tipo de red neuronal GPT-3
- 175 millones de parámetros
- 355 años para entrenar usando una sola GPU
- 4.000,000\$ para entrenar usando una sola GPU



ChatGPT is at capacity right now

[Get notified when we're back](#)

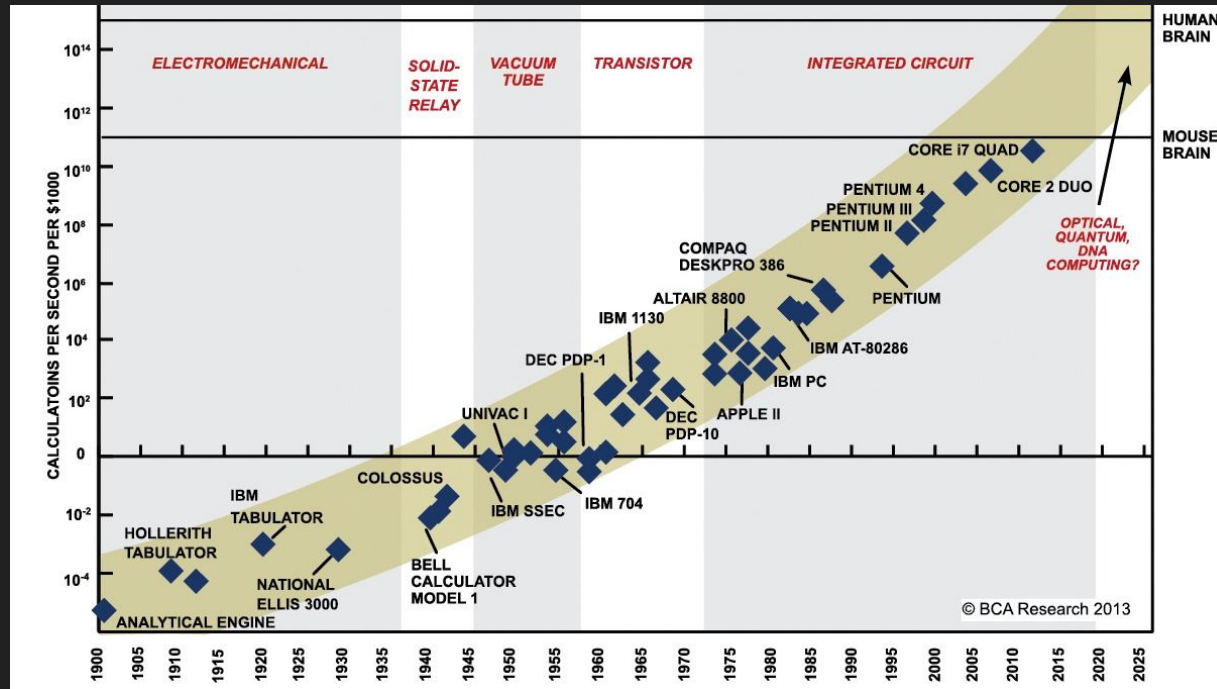
Explain the status of ChatGPT as a pirate.

Ahoy mateys! We be havin' a lot of scurvy dogs checkin' out ChatGPT right now, and we be doin' our best to make sure every landlubber gets a chance to give it a try. We be sorry for any inconvenience this be causin', but we promise it'll be worth the wait. Hang tight, and check back soon, ye swashbucklin' scallywags! We be eager to see what ye think of our chatbot. Arrr!

Ley de Moore

Cada dos años se duplicará el número de transistores en los microchips

George Moore, 1965



Computación de Altas Prestaciones

Entonces... ¿Para qué necesitamos la computación de altas prestaciones?

► 10 Años atrás... **Era del Big Data**

- No teníamos los datos suficientes (ni recursos para obtenerlos ni manejarlos)

► Era actual: Ya tenemos suficientes datos, hay que procesarlos:

- Necesidad de las Redes Neuronales y del HPC... **Master actual!**



Tipos de Aplicaciones

► HTC (High Throughput Computing)

- Aumentar el número de ejecuciones por unidad de tiempo
- Medición en: trabajos/s, trabajos/min, etc.



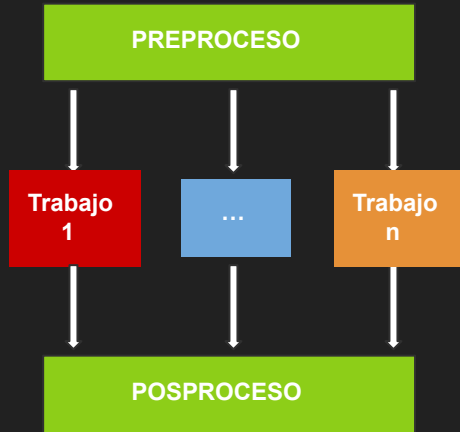
► HPC (High Performance Computing)

- Máximo aprovechamiento de los recursos del sistema
- **Explotar el paralelismo** aprovechando todos los recursos disponibles
- Medición en:
 - Mops/s (Operaciones en memoria por segundo)
 - Flops/s (Operaciones en coma flotante por segundo)
 - MegaFlops 10^6
 - GigaFlops 10^9
 - TeraFlops 10^{12}
 - PetaFlops 10^{18}
 - ...



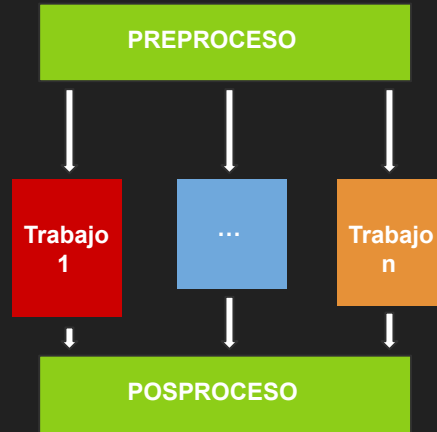
Tipos de Aplicaciones HTC

Síncrono



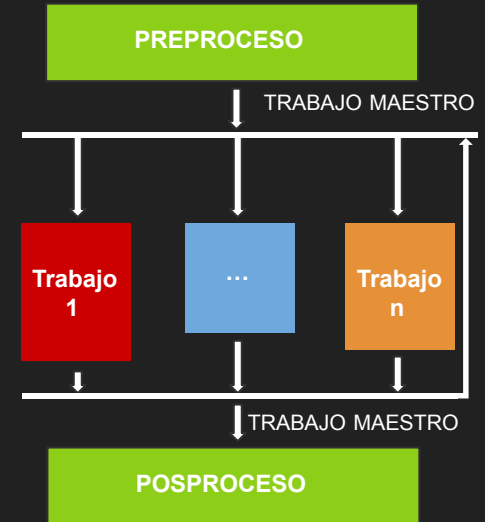
- Trabajos sincrónicos
- Finalizan en el mismo instante de tiempo

Asíncrono



- Trabajos asincrónicos
- Finalizan en diferentes instantes de tiempo

Maestro-Eslavo



- El proceso principal sincroniza el resto de trabajo



Tipos de Aplicaciones HPC



HPC ¡No solo consiste en la ejecución paralela del código!

- Aplicaciones optimizadas
- Uso correcto de las memorias cachés L1, L2 y/o L3 (según arquitectura)
- Técnicas avanzadas de optimización de código
 - Desenrollado de bucles
 - Prefetch a nivel de caché
 - Prefetch a nivel de registros vectoriales
- Ayuda del compilador. Compilación mediante: **-O3, -O2, ...**
- Si se complica la cosa... Compilación mediante la opción: **-S -fverbose-asm**
¡Generación de código en ensamblador para revisarlo!



Tipos de Aplicaciones HPC (Break!)

Ejemplo



```
float *b = (float *) malloc (sizeof(float) * buffer_size);
size_t result = 0;

for (int i= 0; i < buffer_size; i++) b[i] = 1.0;
for (i=0; i < buffer_size; i++) result += b[i];
```

1. gcc -S -fverbose-asm -O0 your_file.c
2. gcc -S -fverbose-asm -O3 your_file.c

Comparar los dos ficheros en ensamblador generados. ¿Qué ocurre?

Tipos de Aplicaciones HPC (Paralelismo a Nivel del Procesador)

Núcleos del procesador

HILOS



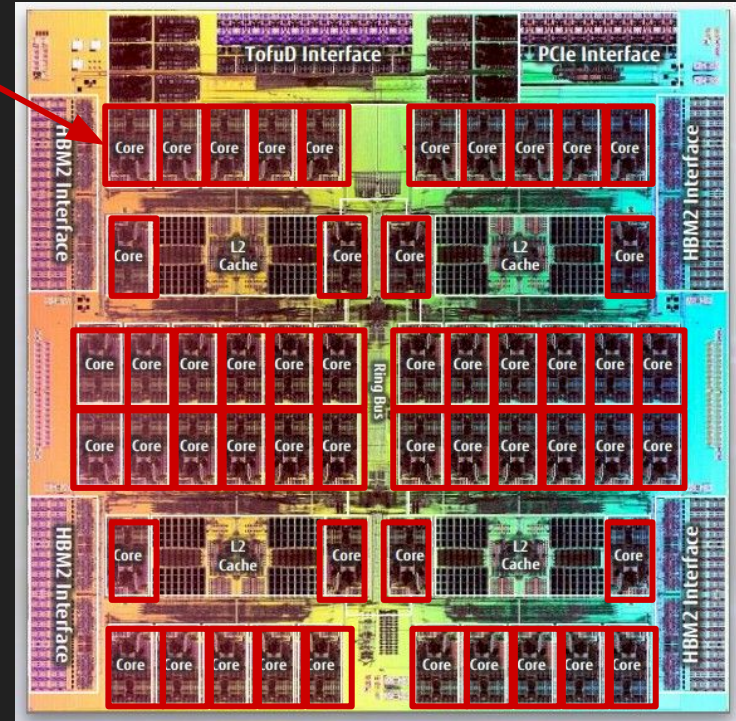
Se ejecutan de forma paralela
A64FX hasta 48 cores disponibles

OBJETIVO 1

Aplicaciones que explotan al máximo el paralelismo del procesador

Ejemplo multiprocesador actual

A64FX FUJITSU



Tipos de Aplicaciones HPC (Paralelismo a Nivel del Procesador)

Pero... ¿Cómo explotamos el paralelismo?

OpenMP

openMP

- Paralelización a nivel de hilos o tareas openMP
`#pragma omp parallel, #pragma omp task,...`

Pthreads POSIX

- Paralelización a nivel de hilos POSIX
`pthread_create(),...`

openMPI

- Paralelización a nivel de procesos MPI
`MPI_Init(),...`

Procesos POSIX

- Paralelización a nivel de procesos POSIX
`fork(),...`



OPEN MPI



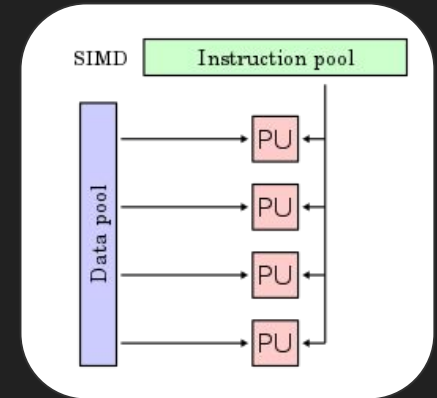
Tipos de Aplicaciones HPC (Paralelismo a Nivel de Datos)

► SIMD: Single Instruction, Multiple Data

Instrucciones que aplican una misma operación sobre un conjunto más o menos grande de datos

► Registros Vectoriales específicos, localizados en el propio núcleo del procesador

► Distinto conjunto de instrucciones y tamaño de registros en función de la arquitectura



ARQUITECTURA	LONGITUD REGISTROS	Floats x Registro	Registros Vectoriales
ARM Neon v8	128 bits	4	32
AMD EPYC (AVX2)	256 bits	8	16
ARM SVE	512 bits	16 ¡Variable!	32
Intel Skylake (AVX-512)	512 bits	16	¿16?

<https://en.wikichip.org/wiki/WikiChip>

Tipos de Aplicaciones HPC (Paralelismo a Nivel de GPU)

HILOS



Se ejecutan de forma paralela

GeForce RTX 4090

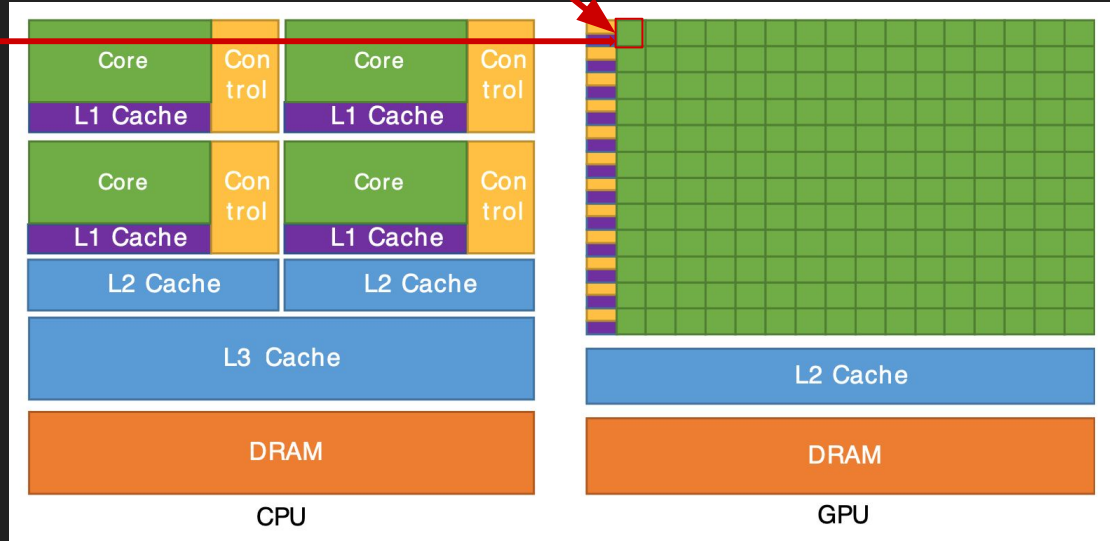
Núcleos NVIDIA CUDA	16384
Frecuencia Reloj	2,52 GHz
Memoria	24 GB



+



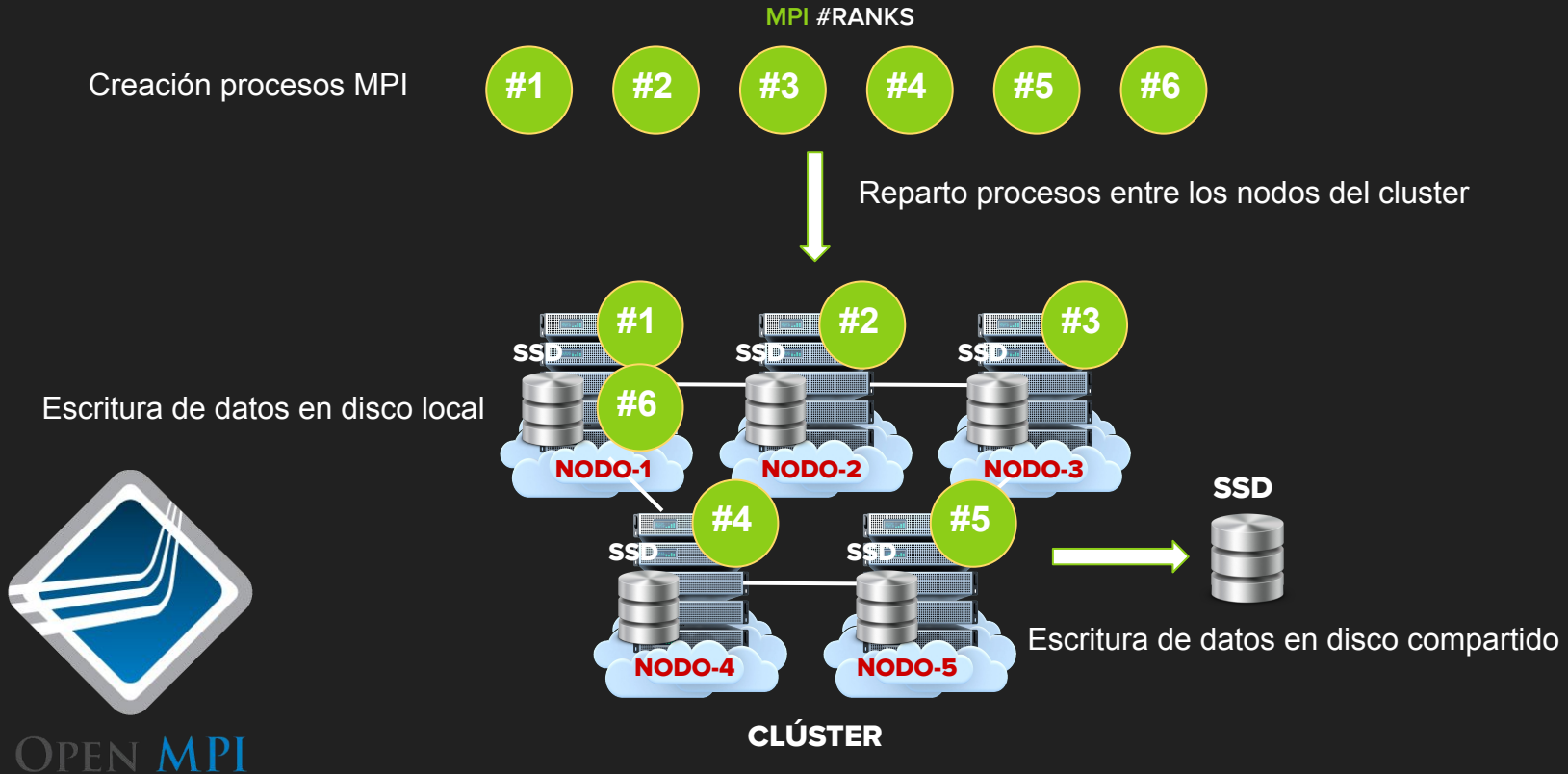
Núcleos de la GPU



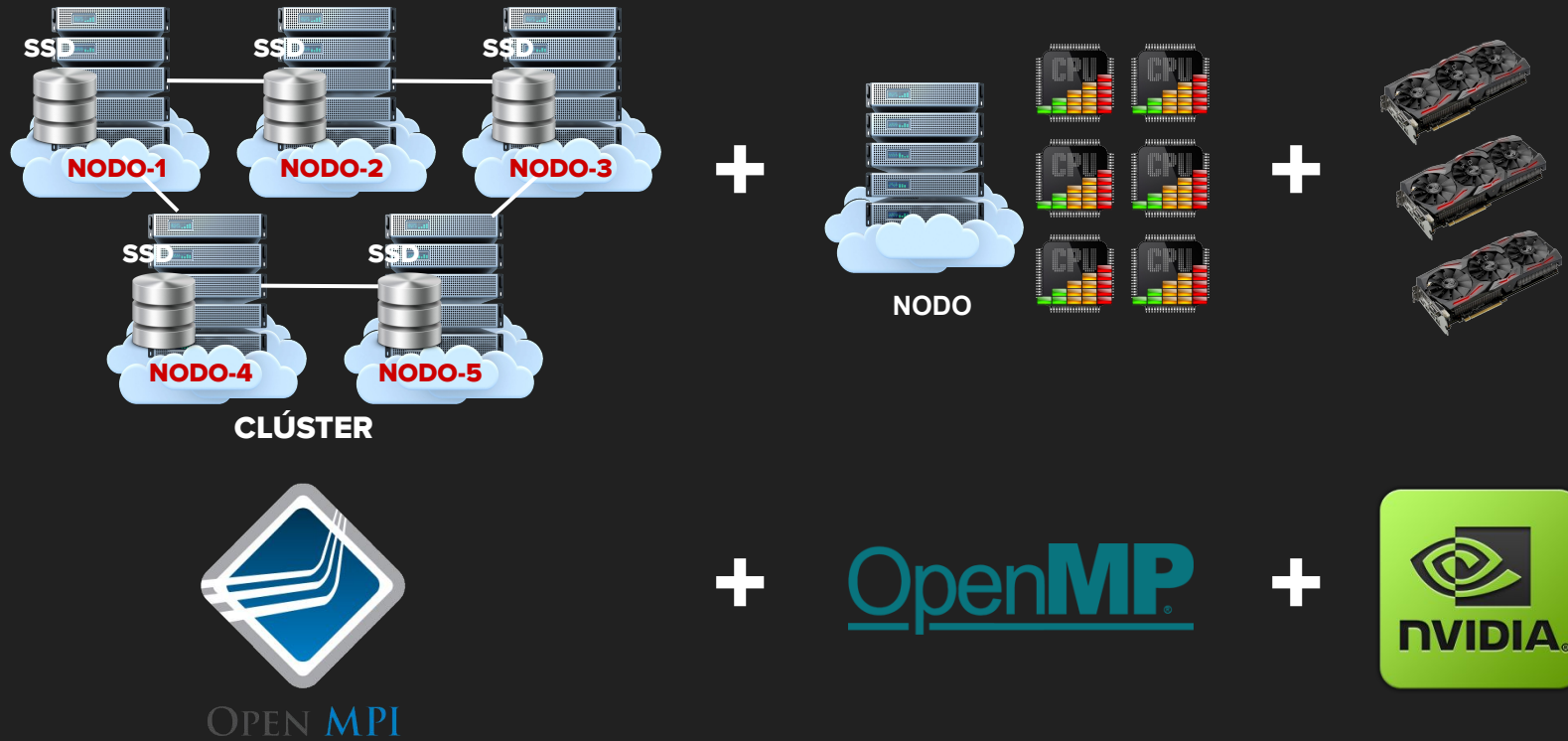
Fuente: Nvidia corp. CUDA C++ Programming Guide: Design Guide. PG-02829-001 v11.6, February 2022



Tipos de Aplicaciones HPC (Paralelismo a Nivel de Clúster)



Tipos de Aplicaciones HPC (Paralelismo Híbrido, Clúster + CPU + GPU)



TOP500 Clústers



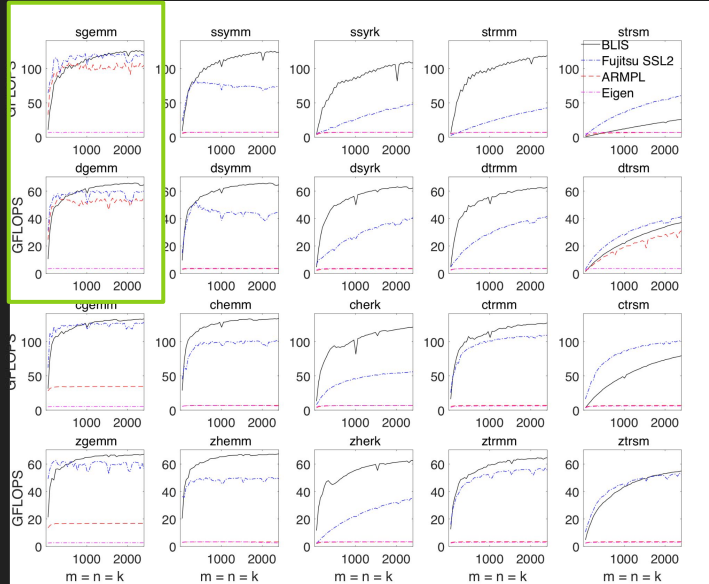
Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,730,112	1,102.00	1,685.65	21,100
2	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
3	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,220,288	309.10	428.70	6,016
4	Leonardo - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, Atos EuroHPC/CINECA Italy	1,463,616	174.70	255.75	5,610

85	Karolina, GPU partition - Apollo 6500, AMD EPYC 7763 64C 2.45GHz, NVIDIA A100 SXM4 40 GB, Infiniband HDR200, HPE IT4Innovations National Supercomputing Center, VSB-Technical University of Ostrava Czechia	71,424	6.75	9.08	311
86	Christofari - NVIDIA DGX-2, Xeon Platinum 8168 24C 2.7GHz, Mellanox InfiniBand EDR, NVIDIA Tesla V100, Nvidia SberCloud Russia	99,600	6.67	8.79	
87	Flow - PRIMEHPC FX1000, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu Information Technology Center, Nagoya University Japan	110,592	6.62	7.79	
88	MareNostrum - Lenovo SD530, Xeon Platinum 8160 24C 2.1GHz, Intel Omni-Path, Lenovo Barcelona Supercomputing Center Spain	153,216	6.47	10.30	1,632

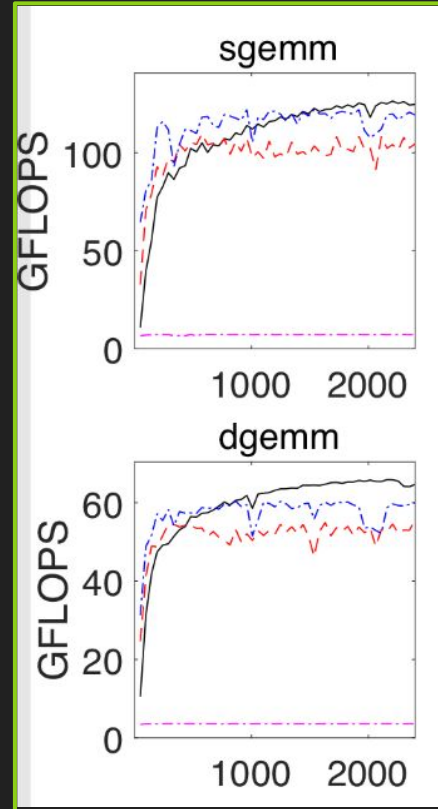


TOP 2 Fugaku: Rendimiento Multiplicación Matrices (GEMM)

GFLOPS GEMM (single-thread)



<https://github.com/flame/blis>



Simple Precisión
140.8 GFlops

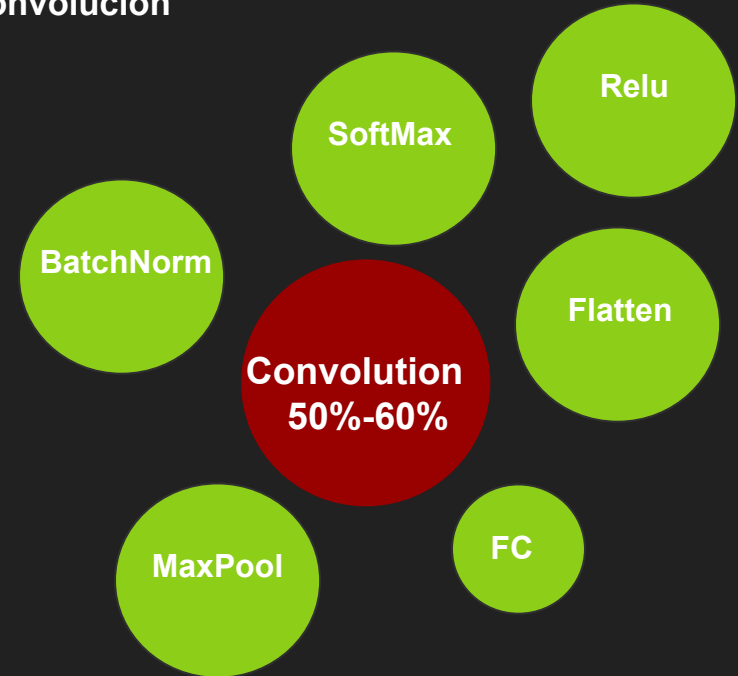
Doble Precisión
70.4 GFlops

¿Por qué GEMM? CNN (Convolutional Neural Network)

Distintos Modelos de Redes Convolucionales



*Entre el **50% - 60%** del tiempo se invierte en la convolución



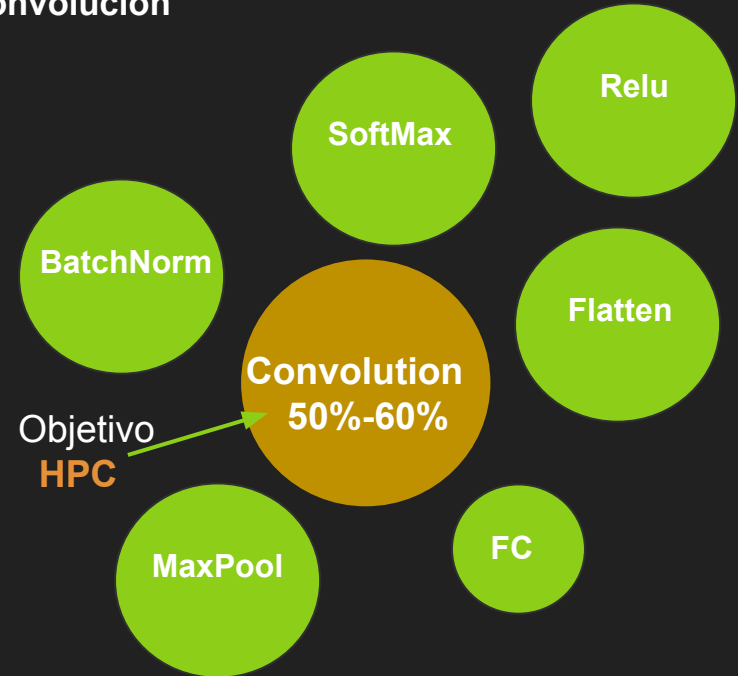
*Puede variar según arquitectura e implementación

¿Por qué GEMM? CNN (Convolutional Neural Network)

Distintos Modelos de Redes Convolucionales



*Entre el **50% - 60%** del tiempo se invierte en la convolución

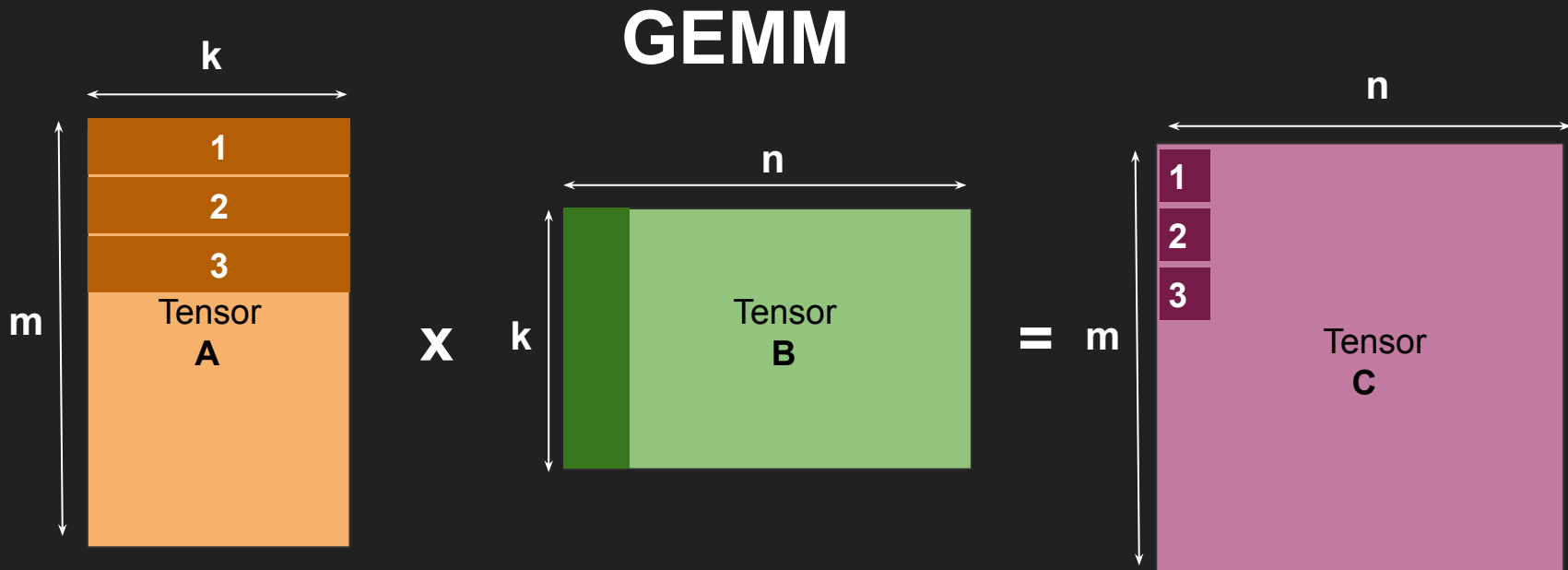


*Puede variar ligeramente según arquitectura e implementación

GEMM en la Convolución

CONVOLUCIÓN = IM2COL / IM2ROW + GEMM

MULTIPLICACIÓN DE MATRICES



- Acceso a los datos de forma localizada en memoria
- Uso de buffers temporales para la preparación de los datos a procesar
- Conocer la arquitectura (tamaño caché L1, L2, L3..., si son compartidas o no...)
- **Prefetching** (técnica usada para traer a memoria caché los próximos datos a procesarse)
- Operaciones aritméticas mediante registros vectoriales
 - **Instrucciones en ensamblador (ASM)**
 - PRFM <prfop>, <addr> | label
 - **Intrínseca**
 - svprfb (svbool_t pg, const void *base, enum svprfop op)

<https://developer.arm.com/documentation/den0024/a/The-A64-instruction-set/Memory-access-instructions/Prefetching-memory>

<https://developer.arm.com/architectures/instruction-sets/intrinsics/#q=prefetch>



Técnicas HPC (Desenrollado de bucles)

- **Ahorrar instrucciones a la hora de procesar bucles** (a veces el propio compilador lleva a cabo esta faena por nosotros)

Bucle Normal

```
for (uint32_t i = 0; i < N; i++)  
    A[i] += B[i];
```

Unroll x4

```
//Siendo 'N' múltiplo de 4  
for (uint32_t i = 0; i < N; i+=4) {  
    A[i+0] += B[i+0];  
    A[i+1] += B[i+1];  
    A[i+2] += B[i+2];  
    A[i+3] += B[i+3];  
}
```

¿Por qué un desenrollado x4?

► SIMD: Single Instruction Multiple Data (Intrínsecas ARMv8)

- 128 bits x Registro Vectorial
- 32 Registros Vectoriales

► Dependiendo del número de datos por registro y del tipo. Ejemplo **FLOATS**:

- **float16x8_t** (8 float de 16 bits por registro)
- **float32x4_t** (4 float de 32 bits por registro)
- **float64x2_t** (2 float de 64 bits por registro)

► Algunas de las intrínsecas:

- **vld1q_f32(float32_t *ptr)** (carga 4 elementos del puntero *ptr a registros)
- **vst1q_f32(float32_t *ptr, float32x4_t reg)** (almacena 4 elementos del puntero *ptr a memoria)
- **vaddq(float32x4_t a, float32x4_t b)** (suma los registros vectoriales a y b)

Ejemplo



Reserva 3 buffers (**A**, **B** y **C**) de tipo simple precisión de 8 elementos

Operar con intrínsecas: $A * B = C$

Comparar resultados

<https://developer.arm.com/architectures/instruction-sets/intrinsics/>

Técnicas HPC (SIMD + Desenrollado de bucles)

Unroll x4

```
//Siendo 'N' múltiplo de 4
for (uint32_t i = 0; i < N; i+=4) {
    A[i+0] += B[i+0];
    A[i+1] += B[i+1];
    A[i+2] += B[i+2];
    A[i+3] += B[i+3];
}
```

Unroll x4 + SIMD

```
//Siendo 'N' múltiplo de 4
float32x4_t _A, _B, _C;
for (uint32_t i = 0; i < N; i+=4) {
    _A = vld1q_f32(&A[i]);
    _B = vld1q_f32(&B[i]);
    _C = vaddq(_A, _B);
    vst1q_f32(&A[i], _C);
}
```

Técnicas HPC (SIMD + Desenrollado de bucles)

Unroll x4

```
//Siendo 'N' múltiplo de 4
for (uint32_t i = 0; i < N; i+=4) {
    A[i+0] += B[i+0];
    A[i+1] += B[i+1];
    A[i+2] += B[i+2];
    A[i+3] += B[i+3];
}
```

DEPENDENCIA
RAW

Unroll x4 + SIMD

```
//Siendo 'N' múltiplo de 4
float32x4_t _A, _B, _C;
for (uint32_t i = 0; i < N; i+=4) {
    _A = vld1q_f32(&A[i]);
    _B = vld1q_f32(&B[i]);
    _C = vaddq(_A, _B);
    vst1q_f32(&A[i], _C);
}
```

Técnicas HPC (SIMD + Desenrollado de bucles + Prefetching)

Unroll x4 + SIMD + SIMD Prefetching

```
//Siendo 'N' múltiplo de 4
float32x4_t _A, _B, _C, _Atmp, _Btmp;

_A = vld1q_f32(&A[i]);
_B = vld1q_f32(&B[i]);

for (uint32_t i = 0; i < N-4; i+=4) {
    _Atmp = vld1q_f32(&A[i]);
    _Btmp = vld1q_f32(&B[i]);

    _C = vaddq(_A, _B);

    _A = _Atmp;
    _B = _Btmp;

    vst1q_f32(&A[i], _C);
}
```



Saxpy: $z = \alpha x + y$

z, x, y: vectores de 32Mb (coma flotante en simple precisión) cada uno
 α : escalar

1. Implementar esta operación en C
2. Temporizar rutina y calcular GFlops
3. Optimizarla con intrínsecas y técnicas HPC
4. Validación y comparativa de tiempos

//Rutina temporización (tiempo en segundos)

```
double dclock() {  
    struct timeval tv;  
    gettimeofday( &tv, NULL );  
    return (double) (tv.tv_sec + tv.tv_usec*1.0e-6);  
}
```

//Cabeceras útiles

```
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/time.h>  
#include <arm_neon.h>
```

//Programa principal

```
int main() {  
    //Declaración y reserva de memoria (z, y, x)  
    double t0 = dclock();  
    //Rutina Saxpy  
    double t1 = dclock();  
    double tsaxpy = t1 - t0;  
    printf("Tiempo rutina saxpy: %.4f\n", tsaxpy);  
}
```

Are You Doing All You Can?

